Southern New Hampshire University

CS 340 Project Two

README

Grazioso Salvare - Rescue Animal Training Dashboard for Austin Animal Center

About the Project

This project provides a fully functional, interactive web application dashboard designed for the international rescue animal training company, Grazioso Salvare. The application interfaces securely with a MongoDB database containing outcomes from the Austin Animal Center. By interacting with the dashboard, users at Grazioso Salvare can easily filter and categorize available dogs to identify the best candidates for specific types of search-and-rescue training (such as Water Rescue, Mountain/Wilderness Rescue, and Disaster/Individual Tracking).

Motivation

The Grazioso Salvare project requires a reliable middleware solution to connect a user-interface dashboard to a MongoDB database. This Python module exists to abstract the complex database queries into simple, reusable methods (like create() and read()), ensuring that the frontend dashboard can efficiently filter and retrieve animal data without needing direct database access.

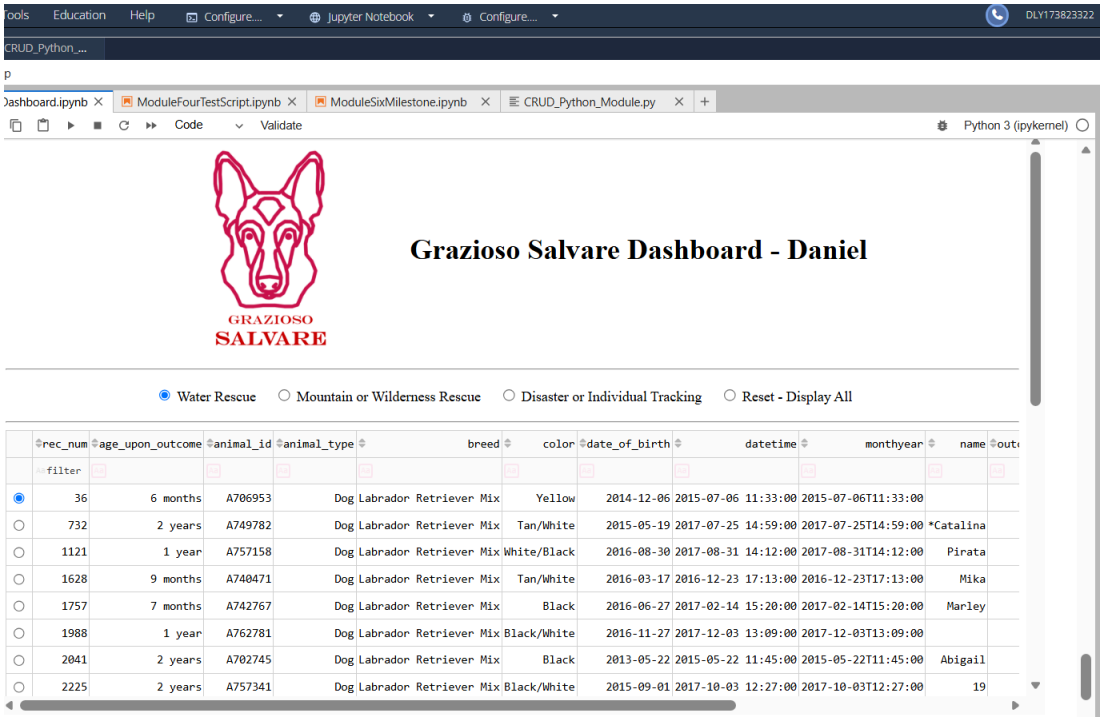
Functionality

The application utilizes a Model-View-Controller (MVC) architecture to provide real-time data visualization. The dashboard features:

Note: This template has been adapted from the following sample templates: Make a README, Best README Template, and A Beginners Guide to Writing a Kickass README.

- An interactive filtering system (radio buttons) that triggers customized database queries based on Grazioso Salvare's required dog profiles (breed, age, and intact status).
- A dynamic, interactive Data Table that updates automatically based on the selected filter. The table supports pagination, native sorting, and single-row selection.
- A dynamic Pie Chart that visualizes the distribution of preferred breeds currently displayed in the data table.
- A Geolocation Map that places a pinpoint on the exact coordinates of the specific animal selected in the data table.

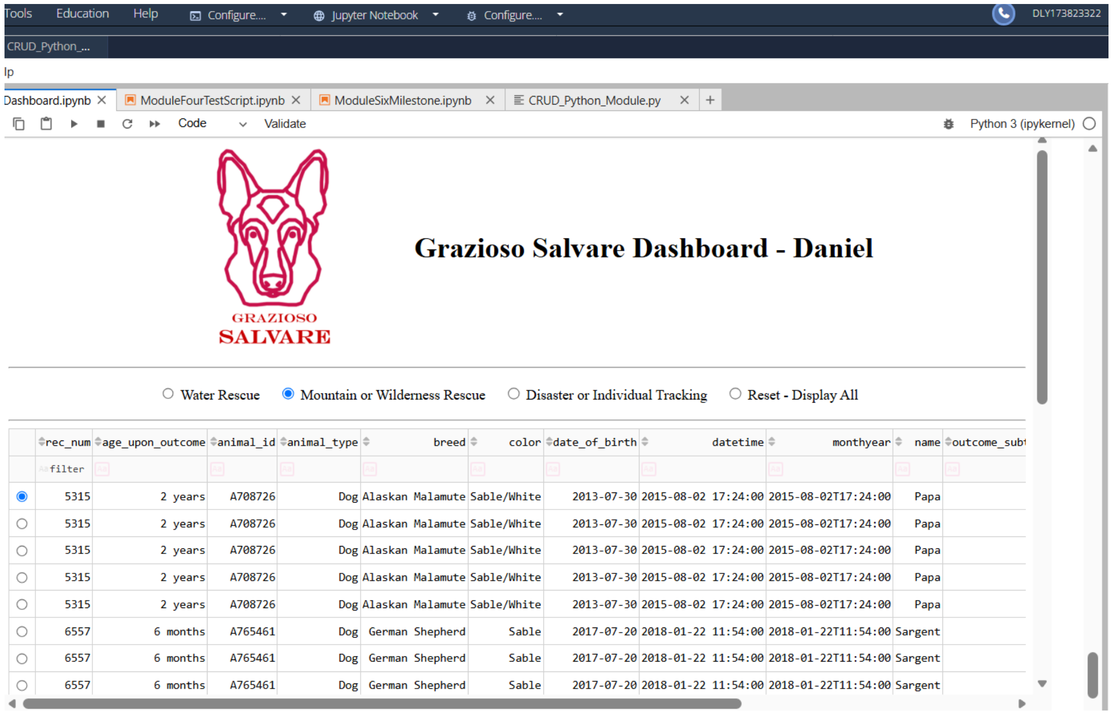
1. Water Rescue Filter:



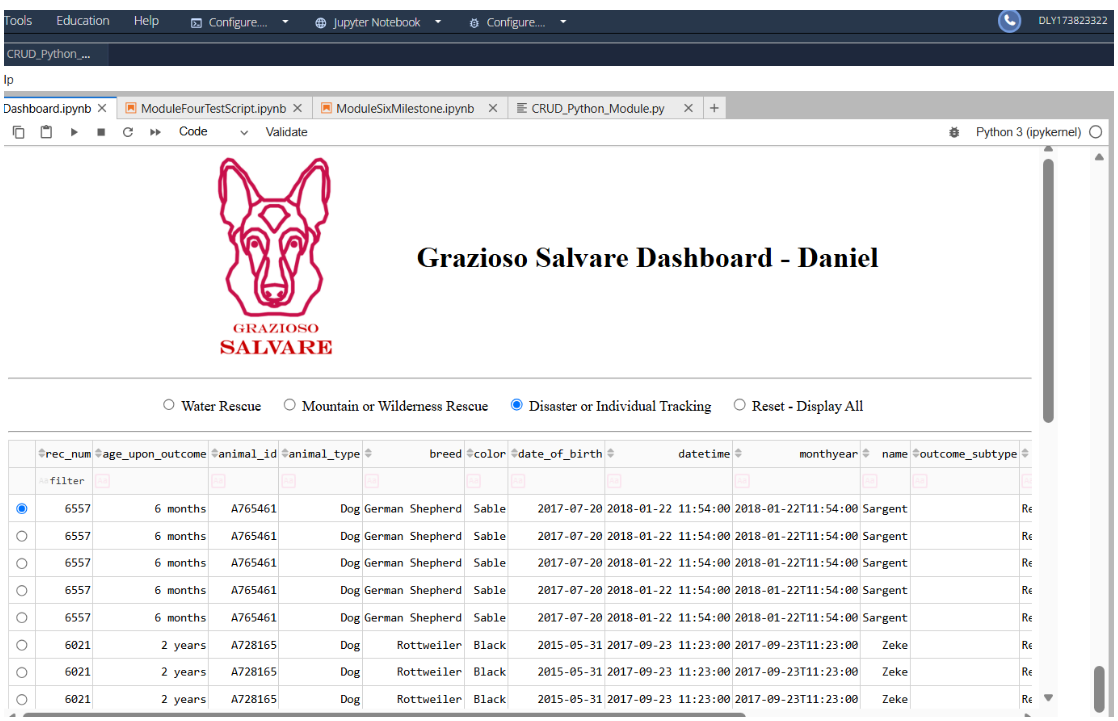
rec_num	age_upon_outcome	animal_id	animal_type	breed	color	date_of_birth	datetime	monthyear	name
36	6 months	A706953	Dog	Labrador Retriever Mix	Yellow	2014-12-06	2015-07-06 11:33:00	2015-07-06T11:33:00	
732	2 years	A749782	Dog	Labrador Retriever Mix	Tan/White	2015-05-19	2017-07-25 14:59:00	2017-07-25T14:59:00	*Catalina
1121	1 year	A757158	Dog	Labrador Retriever Mix	White/Black	2016-08-30	2017-08-31 14:12:00	2017-08-31T14:12:00	Pirata
1628	9 months	A740471	Dog	Labrador Retriever Mix	Tan/White	2016-03-17	2016-12-23 17:13:00	2016-12-23T17:13:00	Mika
1757	7 months	A742767	Dog	Labrador Retriever Mix	Black	2016-06-27	2017-02-14 15:20:00	2017-02-14T15:20:00	Marley
1988	1 year	A762781	Dog	Labrador Retriever Mix	Black/White	2016-11-27	2017-12-03 13:09:00	2017-12-03T13:09:00	
2041	2 years	A702745	Dog	Labrador Retriever Mix	Black	2013-05-22	2015-05-22 11:45:00	2015-05-22T11:45:00	Abigail
2225	2 years	A757341	Dog	Labrador Retriever Mix	Black/White	2015-09-01	2017-10-03 12:27:00	2017-10-03T12:27:00	19

Note: This template has been adapted from the following sample templates: Make a README, Best README Template, and A Beginners Guide to Writing a Kickass README.

2. Mountain or Wilderness Rescue Filter:

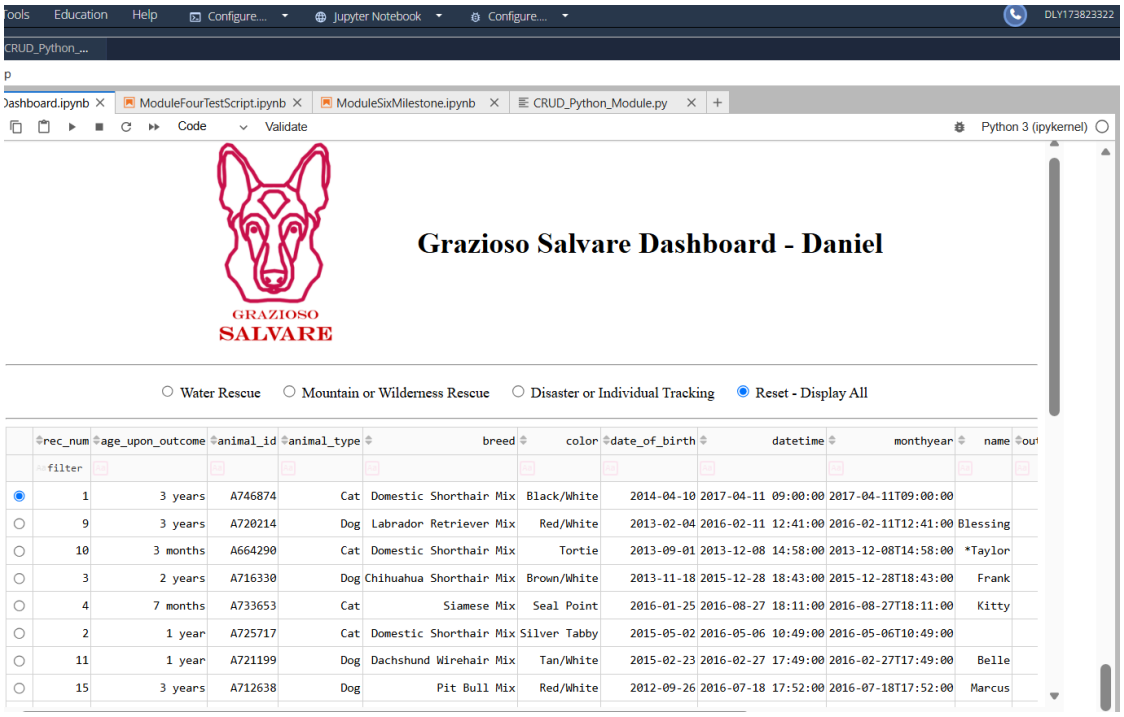


3. Disaster or Individual Tracking Filter:

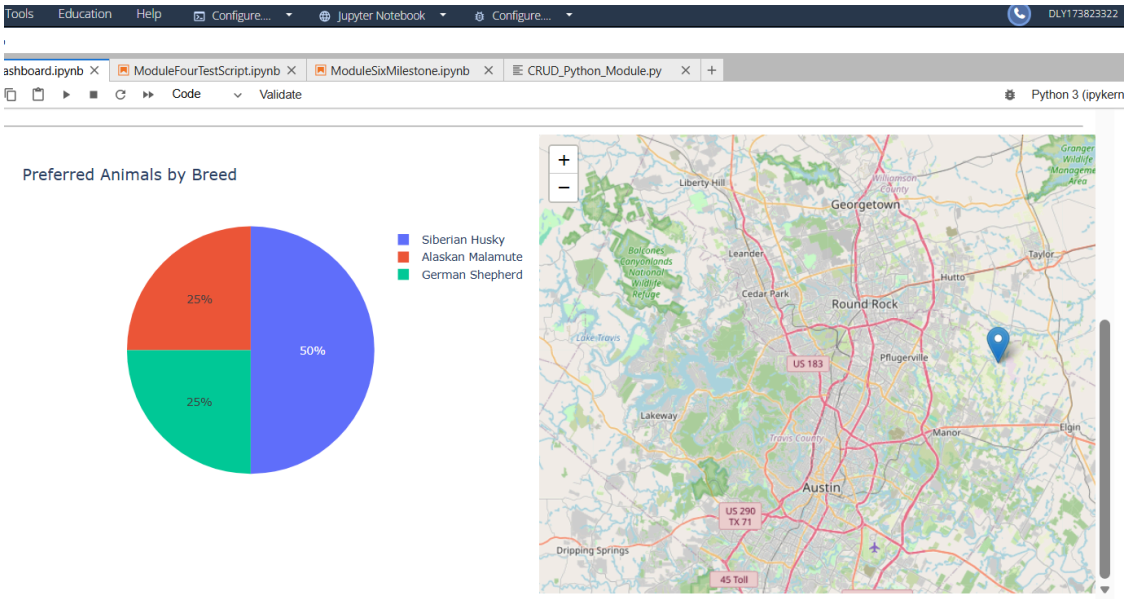


Note: This template has been adapted from the following sample templates: [Make a README](), [Best README Template](), and [A Beginners Guide to Writing a Kickass README]().

4. Reset:



5. Pie chart and geolocation chart



Note: This template has been adapted from the following sample templates: [Make a README](), [Best README Template](), and [A Beginners Guide to Writing a Kickass README]().

Tools Used and Rationale

This application was built using the following tools:

- MongoDB (model): MongoDB was used as the backend database. Because it is a NoSQL, document-oriented database, it provides exceptional flexibility when dealing with unstructured or semi-structured data like the CSV files provided by the animal shelter. Its native integration with BSON/JSON formats maps perfectly to Python dictionaries, making data manipulation highly efficient. https://docs.mongodb.com/ (MongoDB Documentation).
- Dash Framework (view and controller): Built on top of Flask, Plotly, and React, Dash is a powerful framework that allows developers to build analytical web applications entirely in Python without needing to write raw HTML or JavaScript. It provides the View through dash_html_components and dash_table, and it handles the Controller logic using @app.callback decorators, which seamlessly pass user inputs to the database and return updated visualizations. https://dash.plotly.com/ (Dash Documentation)
- JupyterDash: Used to run the Dash application inline within the Jupyter Lab environment for rapid testing.
- PyMongo & Pandas: Used to interface with the database, execute CRUD operations, and format the resulting cursors into structured DataFrames for the dashboard. https://pandas.pydata.org/docs/ (Pandas Documentation)

Steps to Taken

To get a local copy up and running, follow these steps:

1. Database Setup: Ensure MongoDB is installed and running locally. Import the aac_shelter_outcomes.csv dataset into a MongoDB database named aac and a collection named animals.

Note: This template has been adapted from the following sample templates: Make a README, Best README Template, and A Beginners Guide to Writing a Kickass README.

2. User Authentication: Create an authenticated user in the MongoDB admin database named aacuser with readWrite privileges for the aac database.
3. Environment Setup: Ensure Python 3.x is installed along with the required dependencies: pymongo, pandas, jupyter-dash, dash, dash-leaflet, and plotly.
4. File Organization: Place the dataset, the CRUD_Python_Module.py file, the ProjectTwoDashboard.ipynb file, and the Grazioso Salvare Logo image in the same working directory.
5. Execution: Open the Jupyter Notebook, restart the kernel to ensure a clean state, and execute the cell. The dashboard will render inline or on the local host port.

Challenges Encountered

During the integration of the database with the dashboard, a few challenges arose:

1. JSON Serialization Errors: Dash DataTables cannot natively render MongoDB's ObjectID data types, causing the application to crash. This was overcome by writing a Python check to drop the _id column from the Pandas DataFrame before passing the data to the Dash layout.
2. Component ID Mismatches: Callbacks were failing to fire due to mismatching or missing HTML component IDs. This was resolved by carefully aligning the Output and Input strings in the @app.callback decorators to exactly match the IDs defined in the layout (e.g., mapping the filter-type RadioItems to the database query logic).
3. Map Rendering with Missing Data: The geolocation map would fail if a selected row was missing latitude/longitude data. This was overcome by wrapping the coordinate extraction in a try/except block, defaulting to the center of Austin, TX if valid coordinates were unavailable.

Southern New Hampshire University

Contact: Daniel Ly

Note: This template has been adapted from the following sample templates: Make a README, Best README Template, and A Beginners Guide to Writing a Kickass README.